

DPP Assignment 3

Simon Vinding Brodersen(rzn875)

December 10, 2025

1 Tree Operations

1.1 Implement *depths*

Below my implementation of the depths can be seen, which seems to work.

```
def depths (steps: []step) : [](i64, i32) =  
  let (arr, values, keep) = unzip3 <| map (\s ->  
    match s  
      case #u -> (-1, -1, false)  
      case #d x -> (1, x, true)  
  ) steps  
  let inc_scan_arr = scan (+) 0 arr  
  let exc_scan_arr = map2 (\a b -> a - b) inc_scan_arr arr  
  let (depth, value, _) = zip3 exc_scan_arr values keep |> filter (\(_, _, b) -> b) |>  
    unzip3  
  in zip depth value
```

Here we have to convert the steps into items of 1 for down and -1 for up operations. Then we can do an exclusive scan and at last remove all the elements which originally corresponded to up items as these do not add new elements to the graph.

If we let N denote the length of steps, then we have a map, a scan, another map and at last a filter, which all work on an array of the full length N . The filter and the scan operation is what is the limiting factor giving us work of $O(N)$ and span of $O(\lg N)$.

Any sequential computation would have to touch all elements at least once, and as such this implementation is work efficient.

1.2 Implement *parents*

Below is my implementation of the parents function.

```

def parents (D: []i64) : []i64 =
  let n = length D
  let depth_tuple = zip (D :> [n]i64) (iota n)
  let parents = loop parents = [0] for i' < n - 1 do
    let i = i' + 1
    let search_d = D[i] - 1
    let search_ds = map (\i -> depth_tuple[i]) (iota i)
    let (_, p) = reduce_comm (\(d1, i1) (d2, i2) ->
      if (d1 == d2) && (d1 == search_d) then
        if i1 > i2 then (d1, i1)
        else (d2, i2)
      else if d1 == search_d then (d1, i1)
      else if d2 == search_d then (d2, i2)
      else (-1, -1)
    ) (0, -1) search_ds
    in parents ++ [p]
  in parents

```

We initialize the parents with the root being itself, and then otherwise we search to the left of us for items with a value less than the current index, where we choose the higher index in case of a tie. The slides did propose a linear search to the left of the current index, but I decided to instead use the `reduce_comm` to search on the left hand side, which might not be beneficial in practise but should reduce the span of the algorithm.

The algorithm runs a sequential loop over all elements (except the root). Inside this loop it runs a map and a `reduce_comm`. The `reduce_comm` only runs on the left items, meaning that the length that is searched is a reduced subset of the full list. However, as an upper bound this would still be over the full array. As such inside the loop we do $O(N)$ work with a span of $O(\lg N)$. We do this within a loop giving total work of $O(N^2)$ and span $O(N \lg N)$.

As hinted to on the slides, one more efficient algorithm would be to sort the depths keeping track of the indices and tie breaking on said indices. This could be done in $O(n \lg n)$ work. Then the sequential loop would just have to use binary search with the same tie breaker. This would lead to work of $O(2n \lg n)$ in total, so my algorithm is not work-efficient.

1.3 Implement *subtree_sizes*

Below my implementation of the *subtree_sizes* can be seen.

```

def subtree_sizes [n] (steps: [n]step) : []i64 =
  let (D, V) = depths steps |> unzip
  let P = parents D
  let n = length D
  let max_depth = reduce i64.max 0 D
  let (res, _) = loop (res, cur_depth) = (copy V, max_depth) while cur_depth > 0 do
    let indices = filter (\i -> D[i] == cur_depth) (iota n)
    let values = map (\i -> res[i]) indices
    let parent_indices = map (\i -> P[i]) indices
    in (reduce_by_index res (+) (0) parent_indices values, cur_depth - 1)
  in map i64.i32 res

```

This approach uses the naive bottom up iteration. Here we have a call to the reduce function, followed by a sequential loop, which runs over the depth of the tree. This loop calls a filter, map

and `reduce_by_index`. The `reduce_by_index` both have work $O(N)$ and span of $O(\lg N)$ both of which runs over arrays of length equal to the entire steps.

As such, the total work of our algorithm is $O(N \times D)$ and span $O(\lg N \times D)$, where D denotes the maximum depth of the tree.

Tests

The functions were tested as shown below and pass these tests.

```
-- ==
-- entry: test_depth
-- input { "d0 d2 d3 u u d5" }
-- output { [0i64, 1, 2, 1] [0i32, 2, 3, 5] }
-- input { "d0 d2 u d3 u d0 d4 u d0 u u d0 d5" }
-- output { [0i64, 1, 1, 1, 2, 2, 1, 2] [0i32, 2, 3, 0, 4, 0, 0, 5] }
-- input { "d1 d2 d3 d4 d5" }
-- output { [0i64, 1, 2, 3, 4] [1i32, 2, 3, 4, 5] }
-- input { "" }
-- output { empty([0]i64) empty([0]i32) }
entry test_depth [n] (inp: string [n]) = input.steps inp |> depths |> unzip

-- ==
-- entry: test_parents
-- input { "d0 d2 d3 u u d5" }
-- output { [0i64, 0, 1, 0] }
-- input { "d0 d2 u d3 u d0 d4 u d0 u u d0 d5" }
-- output { [0i64, 0, 0, 0, 3, 3, 0, 6] }
-- input { "d1 d2 d3 d4 d5" }
-- output { [0i64, 0, 1, 2, 3] }
-- input { "" }
-- output { empty([0]i64) }
entry test_parents [n] (inp: string [n]) =
  let (D, _) = input.steps inp |> depths |> unzip
  in parents D

-- ==
-- entry: test_subtree_sizes
-- input { "d0 d2 u d3 u d0 d4 u d0 u u d0 d5" }
-- output { [14i64, 2, 3, 4, 4, 0, 5, 5] }
-- input { "d1 d2 d3 d4 d5" }
-- output { [15i64, 14, 12, 9, 5] }
-- input { "" }
-- output { empty([0]i64) }
entry test_subtree_sizes [n] (inp: string [n]) = input.steps inp |> subtree_sizes
```

The tests can be run by running `futhark test trees.fut`.

2 Region Labeling

2.1 Naive region labeling

My implementation of the naive algorithm can be seen below.

```

def region_label_naive [h] [w] (img: [h][w]u32) : [h][w]i64 =
  let labels_flat = flatten (tabulate_2d h w \i j -> flat_pos w (i, j))
  let (_, res) =
    loop (prev_labels, current_labels) = (replicate (h * w) (-1), labels_flat)
    while prev_labels != current_labels do
      let new_labels =
        map (\fpos ->
          let pos = unflat_pos w fpos
          let color = get pos img
          let neighbours =
            map (\d ->
              let npos = move d pos
              in if in_bounds img npos
                then (npos, get npos img)
                else (no_pos, 0))
              [#n, #w, #e, #s]
          let same_color_labels = map (\(npos, c) -> if c == color && npos != no_pos
            then current_labels[flat_pos w npos] else -1i64) neighbours
          in reduce i64.max current_labels[fpos] same_color_labels)
          labels_flat
        in (current_labels, new_labels)
  in unflatten (res :> [h * w]i64)

```

The implementation maps over all the pixels in the images. It checks which neighbours have the same color, and the neighbours without the same color are set to -1, such that they are not chosen in the reduce.

Inside the loop we have a map function, which runs over all pixels in the image. Inside the map, we do a constant amount of work (although we do have a map it is always over the neighbours and the same with the reduce). As such the inside of the loop has work $O(N)$ and span $O(1)$. However, figuring out how many times the loop runs is a little more tricky.

If the entire image is all the same color, then we will have to propagate the largest label throughout the entire image. This label would be in the bottom right corner of the image. At each iteration it moves one pixel in each direction, so before reaching the top left corner, it would have to do $H + W$ iterations, where H is the height and W is the width. As the loop is sequential, this gives us a total work of $O(N \times (H + W))$ and span of $O(H + W)$. Note that $H + W - 1 \leq H * W = N^1$ as both H and W are greater or equal to 1.

2.2 Enlightened region labeling

Below my code for the algorithm can be seen.

¹the -1 comes as in the example before we should not count the bottom left corner twice.

```

def norm_edge' (a: i64, b: i64) : (i64, i64) =
  if a <= b then (a, b) else (b, a)

-- / Create normalised edges linking all neighbouring pixels with the same
-- colour.
def mk_edges [h] [w] (img: [h][w]u32) : ?[k].[k]edge =
  let directions = [#n, #w, #s, #e]
  let edges =
    tabulate_2d h w (\r c ->
      let pos = (r, c)
      let color = get pos img
      let edges =
        map (\d ->
          let neighbour = move d pos
          in if in_bounds img neighbour
            && color == (get neighbour img)
            then norm_edge w (pos, neighbour)
            else (no_pos, no_pos))
          directions
        in edges)
    |> flatten_3d
    |> filter (\(a, b) -> a != no_pos && b != no_pos)
  in edges

def region_label_enlightened [h] [w] (img: [h][w]u32) =
  -- Step 1: compute edges.
  let edges = map (\(a, b) -> (flat_pos w a, flat_pos w b)) (mk_edges img)
  -- Step 2: Initialise DAG.
  let forest = flatten (tabulate_2d h w \i j -> flat_pos w (i, j))
  let (forest', _) =
    loop (forest, edges) while length edges > 0 do
      let possible_edges = filter (\(u, _) -> forest[u] == u) edges
      let sources = map (.0) possible_edges
      let targets = map (.1) possible_edges
      let new_forest = reduce_by_index forest i64.max (-1) sources targets
      let non_inserted_edges = filter (\(a, b) -> new_forest[a] != b) edges
      let new_edges = map (\(a, b) -> norm_edge' (new_forest[a], new_forest[b]))
        non_inserted_edges
      in (new_forest, new_edges)
  in map (\a' -> loop a = a' while a != forest'[a] do forest'[a]) forest' |> unflatten

```

The algorithm first computes the initial edges via the `mk_edges` function. This function calls `tabulate_2d`, which has work $O(h \times w \times f)$ and span $O(S(f))$ where f is the function we run. In our case, this function has a constant work and span as we always map over 4 directions, and the rest of the operations are also constant work and span. Therefore, we have $O(h \times w) = O(n)$ work and $O(1)$ span for the `tabulate_2d`. We then call `flatten` and `filter`, where the `filter` dominates our total work and span giving us a work of $O(n)$ and span of $O(\lg n)$.

Back inside the main function, we call a `map`, `flatten` and `tabulate`, all of which are dominated by `mk_edges` found before, and so provide nothing to the complexity.

Now comes the sequential loop. To argue about the number of iterations one has to think about how the "length" of the edges are updated. When we first initialize all the edges, the edges only point between neighbours. However, after one iteration the edges move to the parents, meaning that the "length" of the edges doubles. Then in the iteration after this, both again move to the parents, which beforehand had updated their edges themselves, meaning that we now have edges

with lengths 4 and so they doubled again. In the next step it will be 8 and so on. If we ever reach a root node, then we insert the node and we are not taking it into account in the following iteration, and by the aforementioned logic the most amount of contractions we can do before reaching a root node is $\lg n$ as the length always doubles. This means, that the loop will at most run $\lg n$ times.

Inside of the loop, we filter on the edges call `reduce_by_idx` on the possible edges, filter again and call a map. The `reduce_by_idx` is only ever called on each original edge once, as once it has been called upon, it is removed from the remaining edges. The filter on the other hand could be rerun on the same edges and in the worst case the length of the edges is n , so the inner loop has work $O(n)$ and span $O(\lg n)$ giving us a total work of $O(n \lg n)$ and span $O(\lg^2 n)$.

Note on testing

Both of my algorithms were simply tested on the provided images. When I run the *futhark* *literate regionlabeling.fut*, it produces matching colored images and they both look like what I would expect, so I believe both algorithms to be functional.